The nukeCE Anti-Drift Doctrine: A Comprehensive Framework for Repository-Resident AI Governance

1. The Entropy of Intelligence: Theoretical Foundations of Repository Drift

The assimilation of autonomous artificial intelligence agents into software repositories marks a definitive transition from static code storage to dynamic, agentic environments. In this emerging paradigm, the repository is no longer merely a passive archive of source code, version history, and disparate documentation; it has become an active operational environment—a "context engine" that informs, constrains, and drives model behavior. However, this transition introduces a critical, systemic vulnerability: entropy. As agents operate within the repository, generating code, interpreting instructions, and modifying files, they are subject to powerful entropic forces that degrade system integrity over time. Without a coherent, enforceable governance doctrine, repository-resident AI systems suffer from a triad of degradation vectors: *configuration drift*, where the infrastructure state diverges from documented intent; *architectural drift*, where the realized system structure violates its design principles; and *semantic drift*, where the generative model's output progressively diverges from factual ground truth.

The "nukeCE" (Nuclear Canonical Enforcement) Doctrine is proposed here as a minimal, enforceable framework designed to counteract these entropic forces. It posits that the stability of a repository-resident AI system is defined not by the sophistication of the underlying Large Language Model (LLM), but by the rigorous maintenance of a **Single Source of Truth (SSoT)**. The doctrine prioritizes interchangeable models—treating the specific AI provider as a transient utility—and elevates the repository's semantic structure to the status of immutable law. This report exhaustively analyzes the mechanisms of drift, the architecture of repository-resident truth, and the enforcement protocols necessary to maintain a "clean-slate" operational environment where bias artifacts are strictly segregated from permissions.

1.1 The Anatomy of Drift in Agentic Systems

Drift is the silent killer of software reliability. In traditional DevOps contexts, configuration drift is defined as the phenomenon where production infrastructure configurations fall out of sync with their expected state.[1] This divergence leads to "mysterious outages, unexpected regressions, and unhappy customers," primarily because manual, ad-hoc changes bypass the established deployment process, creating a "snowflake" infrastructure that is impossible to reproduce.[1] In the cloud era, this is exacerbated by the ease with which users can manually

alter configurations through management consoles, creating discrepancies between the Infrastructure as Code (IaC) definitions and the live environment.[2]

However, when an AI agent is introduced into the repository, the definition of drift expands significantly. We must now contend with *architectural drift* and *semantic drift*.

1.1.1 Architectural Drift: The Invisible Risk

Architectural drift occurs when an application's business logic, microservice boundaries, or data flows evolve in the code but remain outdated in the documentation or the AI's system prompt.[3] This is a pervasive issue in rapidly evolving codebases. As technological innovation progresses, organizations expand their infrastructure, and without rigorous enforcement, the implementation "drifts" out of its intended configuration or approved operating boundaries.[3]

CrowdStrike researchers note that architectural drift is particularly dangerous because it introduces risks that are often invisible until a catastrophic incident occurs. For instance, an AI agent might analyze a service and, based on outdated context, assume it is stateless. Consequently, the agent might generate error-handling code that fails to account for state persistence, introducing a race condition or data corruption bug that remains dormant until high load conditions trigger it. The agent has not failed in its coding task; it has failed because the architecture it perceives (via the repository context) no longer matches the architecture that exists (in the production environment). This highlights the necessity of **Application Security Posture Management (ASPM)** approaches that can baseline and lock in application architectures, alerting teams to drift in real-time.[3]

1.1.2 Semantic Drift: The Hallucination of Coherence

Perhaps the most insidious form of entropy in AI systems is *semantic drift*. In the context of generative models, semantic drift describes the phenomenon where the quality of text generation decreases as the length of the generation increases.[4] Research on Wikipedia-style biography generation has explicitly shown that modern LLMs tend to generate correct facts first, but then "drift away" into incorrect facts, hallucinations, or irrelevant content as the sequence progresses.[4]

This drift is quantified by the "semantic drift score," which measures the degree of separation between correct and incorrect facts. A high drift score indicates that the model has switched from retrieving knowledge to fabricating it. In a repository context, semantic drift is exacerbated by "context rot".[6] As the AI agent is fed a "file-cabinet" of disorganized, redundant, or obsolete documentation, its attention mechanism becomes diluted. The model must expend its finite token budget processing "noise"—outdated READMEs, contradictory wiki pages, and deprecated API references. This dilution of signal leads to a loss of coherence, relevance, and truthfulness.[4]

The consequences of semantic drift in a code repository are severe. An agent suffering from

semantic drift might generate code that is syntactically perfect but semantically incoherent with the project's goals—for example, implementing a feature using a library that was deprecated three versions ago because the agent's context window was polluted with old migration guides. Thus, the nukeCE doctrine asserts that **drift management is not merely a maintenance task; it is the primary security and reliability constraint for agentic systems.**

1.2 The Economic Physics of Context Windows

To understand why drift is unavoidable without strict governance, one must analyze the "token economics" of LLMs. The context window—the amount of text an AI can process at one time—is a finite, scarce resource. While modern models boast windows of 100,000 or even 1 million tokens, the *effective* context window is much smaller due to the "Lost in the Middle" phenomenon and the degradation of attention over long sequences.[6]

Research indicates that as the number of tokens in the context window increases, the model's ability to accurately recall specific information decreases.[6] This is "context rot." When a repository allows "file-cabinet sprawl"—the accumulation of redundant, uncurated documents—it forces the agent to consume its token budget on low-value data. This has two economic impacts:

1. **Direct Cost:** Processing more tokens costs more money.
2. **Opportunity Cost:** Tokens spent reading a verbose, outdated "Getting Started" guide are tokens that cannot be used to analyze the complex stack trace of a current bug.

The nukeCE doctrine treats the context window as a highly constrained economic zone. Every file admitted into the context must pay "rent" in the form of high signal-to-noise ratio. If a file duplicates information found elsewhere, it is "evicted" (deleted or archived) to preserve the semantic integrity of the agent's working memory.[8]

2. The Doctrine of the Single Source: Repository-Resident Truth

The antidote to the entropy of drift is the rigorous enforcement of the **Single Source of Truth (SSoT)**. In the nukeCE framework, SSoT is not an abstract concept; it is a concrete architectural mandate that governs file structure, documentation practices, and data management.

2.1 One Authoritative Place: The Anti-Sprawl Mandate

The philosophy of "One Authoritative Place" is derived from the "Docs as Code" movement, which argues that documentation should be stored, versioned, and tested alongside the source code it describes.[10] This alignment ensures that the documentation exists in the same

version control repository as the codebase, using the same workflow.[10]

However, simply co-locating docs and code is insufficient if the documentation itself is fragmented. "File-cabinet sprawl" occurs when information is duplicated across multiple locations: a README.md, a CONTRIBUTING.md, a Wiki, a Google Doc linked in a Slack channel, and code comments. This redundancy creates a "split-brain" scenario for the AI agent. If the README says the database port is 5432, but the CONTRIBUTING guide says 5433, the AI cannot resolve the conflict deterministically.

Doctrine Principle 2.1: *Information must exist in exactly one location. All other references must be pointers.*

The nukeCE doctrine mandates a **Pointer-Only Architecture**. This concept, borrowed from data management and C programming, dictates that data (or truth) should be stored once, and all other needs for that data should be met by referencing the address (pointer) of the original.[12] In a repository, this means:

- **No Copy-Paste:** If a deployment step is documented in docs/deployment.md, it must never be repeated in the root README.md.
- **Symlinks and Transclusion:** The README.md should use a symbolic link (symlink) or a Markdown transclusion directive to display the content of docs/deployment.md. If the docs/deployment.md file is updated, the README.md reflects the change instantly and automatically.[14]
- **The SSoT Principle:** The repository acts as a normalized database. Data redundancy is treated as a critical bug, equivalent to a memory leak in code.[15]

2.2 Semantic De-duplication: The War on Clones

Semantic duplication is subtler and more dangerous than exact duplication. It involves "Type-4 clones," where code or text is functionally identical but syntactically different.[15] For example, a repository might contain two functions: getUserData(id) and fetchUserInfo(userId), both performing the same SELECT query but written by different developers. Similarly, documentation might contain "How to Install" and "Setup Guide," both describing the same process with different words.

For a human, this is annoying. For an AI agent, this is a source of profound confusion. The agent must expend compute resources to analyze both versions, determine if they differ in subtle ways, and decide which one to follow. This "decision friction" increases the probability of hallucination.[17]

Doctrine Principle 2.2: *Semantic duplication is a build failure.*

The repository must employ semantic linting tools that utilize vector embeddings to detect semantic overlaps.[17] Tools like SemDeDup or DG Clone analyze the vector similarity of text

blocks. If two blocks have a cosine similarity exceeding a defined threshold (e.g., 0.85), they are flagged as potential duplicates. The doctrine requires immediate consolidation: the superior version is retained, and the inferior version is deleted and replaced with a pointer to the master copy.[19]

2.3 Living Documentation vs. Immutable Records

A common point of failure in repository management is the confusion between "living" documentation (which evolves) and "immutable" records (which preserve history). The nukeCE doctrine enforces a strict separation between the two.

2.3.1 The Immutable Log: Architectural Decision Records (ADRs)

To prevent "Architectural Drift," where the reasoning behind a decision is lost, the repository must maintain an immutable log of Architectural Decision Records (ADRs).[21] An ADR captures a decision, its context, and its consequences at a specific point in time.[22]

- **Immutability:** Once an ADR is approved and merged, it is immutable. It serves as a historical fact. "On January 15, 2024, we decided to use PostgreSQL."
- **Supersession:** If the team later decides to switch to MongoDB, they do not edit the original ADR. They create a new ADR ("Migrate to MongoDB") which explicitly marks the previous ADR as "Superseded".[23]
- **Agent Interaction:** AI agents are instructed to treat ADRs as "case law." They provide the *why* behind the current state. When an agent proposes a refactor, it must verify that its proposal does not violate active ADRs.[22]

2.3.2 The Living State: AGENTS.md

In contrast, the instructions for the AI agent itself—stored in AGENTS.md—are "living" documents. They represent the *current* constitution of the repository. If the build system changes from Maven to Gradle, AGENTS.md is overwritten to reflect the new reality. The agent reads this file to know *what to do now*.[24]

3. The Architecture of Anti-Drift: Artifacts and Standards

To operationalize the nukeCE doctrine, a specific file hierarchy is required. This hierarchy is designed to be machine-readable first, human-readable second. It creates a structured "context map" that allows the agent to navigate the repository with minimal token usage.

3.1 The Canonical Context Map: llms.txt

Just as search engines rely on robots.txt to navigate the web, repository-resident AI agents require a standardized entry point to understand the repository's topography. The nukeCE

doctrine mandates the use of llms.txt.[25]

Table 1: Comparison of robots.txt vs. llms.txt

Feature	robots.txt	llms.txt
Primary Audience	Web Crawlers (Googlebot, Bingbot)	LLMs, AI Agents, Coding Assistants
Function	Exclusionary (Block access)	Inclusionary/Curatorial (Guide to value)
Logic	Directives (Disallow: /admin)	Pointers (-(docs/api.md): Core API)
Goal	Prevent indexing of private/low-value pages	Optimize inference context with high-signal data
Format	Plain Text Directives	Markdown with structured headers

Source: Analysis of [27]

The llms.txt file sits at the root of the repository. It is the "treasure map" for the agent.[28] It does not contain content itself; it contains pointers to the content.

- **Structure:** The file uses a Markdown format.[30]
  - **H1 Header:** Project Name.
  - **Blockquote:** A concise summary of the project's purpose ("Context Injection").
  - **Sections:** Categorized lists of links to "Core Documentation," "Source Maps," and "Conventions."
  - **Optional Section:** A specific section marked "Optional" containing links to secondary resources (e.g., deep-dive tutorials) that the agent can skip if its context window is constrained.[26]

This file solves the "File-Cabinet Sprawl" problem by forcing developers to curate the information presented to the AI. If a document is not linked in llms.txt, it effectively does not exist for the agent.[31]

3.2 The Dynamic Repository Map: repomap

While llms.txt handles static documentation, the codebase itself is a dynamic entity that

changes with every commit. To prevent "Ghost Code" hallucinations—where the agent generates code based on an outdated understanding of the file structure—the doctrine requires a dynamic repomap.[32]

- **Generation:** The repomap is generated automatically using tools that parse the Abstract Syntax Tree (AST) of the code (e.g., using tree-sitter).[33] It extracts class names, function signatures, and key variable definitions.
- **Ranking:** Because a full AST dump would overwhelm the context window, the repomap utilizes a graph ranking algorithm (similar to PageRank). It identifies the most "central" or "referenced" modules in the codebase and prioritizes them in the map.[33]
- **Token Optimization:** The generation process respects a strict token budget (e.g., 1,024 tokens). It packs the most critical structural information into this budget, ensuring the agent has a high-level overview of the system without reading every file.[33]
- **Automation:** Crucially, the nukeCE doctrine requires that this map be regenerated via a Git pre-commit hook. If the code changes, the map changes. This ensures the AI always perceives the *current* state of the code.[34]

3.3 The Agent Constitution: AGENTS.md

The AGENTS.md file serves as the explicit instruction set for the AI agent.[35] It acts as the "README for robots".[24]

- **Role:** It decouples instructions from the model. Instead of relying on a custom "System Prompt" configured in the AI provider's dashboard (which creates vendor lock-in), the instructions live in the repository.[24] This ensures **Model Interchangeability**. The repository owner can switch from GPT-4 to Claude 3.5 Sonnet, and the new model simply reads AGENTS.md to understand the rules of engagement.[36]
- **Hierarchy:** The doctrine supports a cascading hierarchy of agent instructions.
  - **Root AGENTS.md:** Defines global constraints (e.g., "Use TypeScript," "Run lint before commit").
  - **Subdirectory AGENTS.md:** Defines local context (e.g., "In /backend, use Python 3.11 and Pytest").[35]
- **Write-Protection:** To prevent the agent from rewriting its own laws to bypass constraints, AGENTS.md must be strictly write-protected. Only a human with elevated permissions can modify the agent's constitution.[35]

4. Interchangeable Models and Bias Management

A core tenet of the nukeCE doctrine is that the AI model is a commodity component, not a foundational partner. The repository must be robust enough to survive the swapping of its intelligence engine. This requires a rigorous distinction between "Bias Artifacts" and "Permissions."

4.1 Bias Artifacts are Preferences, Not Permissions

In AI alignment theory, a "bias artifact" refers to a preference or tuning parameter that influences the *style* or *focus* of the generation.[37] "Permissions," conversely, are hard constraints regarding what the agent is allowed to do.[38] Drift occurs when these two concepts are conflated in the system prompt.

Doctrine Principle 4.1: *Bias artifacts are interchangeable preferences. Permissions are architectural laws.*

- **Soft Preferences (Bias):** These are instructions that guide the *style* of the output. Examples include "Prefer functional programming patterns," "Use concise error messages," or "Write detailed docstrings".[39] These belong in AGENTS.md. They are mutable and can be tuned to the specific strengths of the current model.
- **Hard Constraints (Permissions):** These are binary pass/fail conditions that define the safety boundaries of the system. Examples include "Must pass npm test," "Must not introduce new dependencies without approval," or "Must not read .env files".[41]

By separating these, the repository remains neutral. If the team switches to a new model that requires more explicit prompting to write good code, they update the "Soft Preferences" in AGENTS.md. The "Hard Constraints"—the repository's immune system—remain untouched because they are enforced by the CI/CD pipeline, not the AI.[40]

4.2 The Interchangeability Layer via Context Mapping

To achieve true interchangeability, the repository uses a "Context Mapping" layer.[42] This abstraction layer translates the repository's generic intent into the specific prompt format required by the current model.

- **Abstract Capabilities:** The repository configuration (e.g., .contextgit/config.yaml) defines abstract capabilities like scan_file, run_test, or generate_id.[43]
- **Runtime Translation:** When the agent operates, a runtime adapter translates these abstract capabilities into the specific tool-calling syntax of the active model (e.g., OpenAI Function Calling vs. Anthropic Tool Use).[44]
- **Benefit:** This prevents "Vendor Drift," where the repository becomes littered with provider-specific prompt hacks (e.g., "IMPORTANT: Think step-by-step" or specific XML tag structures tailored to one model). The repository data remains clean, neutral, and model-agnostic.[24]

5. Enforceable Protocols: The Repository Immune System

A doctrine is only as good as its enforcement. "nukeCE" does not rely on the AI's "willingness"

to follow rules; it relies on "Gitops" and automated hooks to police the repository.

5.1 The "Pre-Commit" Immune System: Git Hooks

Git hooks are scripts that run locally before a commit is finalized. In the nukeCE doctrine, they serve as the first line of defense, rejecting "infected" (drifted) code before it can enter the repository history.[45]

Table 2: Mandatory Git Hooks for nukeCE

Hook Name	Function	Drift Vector Addressed	Failure Consequence
Context Integrity	Scans llms.txt & docs for broken pointers	"Pointer Rot" / Context Rot	Commit Rejected [46]
Map Regenerator	Re-runs repomap generation tool	Ghost Code / Hallucination	Map Updated Automatically [32]
Semantic Linter	Checks for text/code duplication (Vale)	Semantic Duplication / Sprawl	Warning/Rejection [47]
Secret Scanner	Scans for API keys/credentials	Permission Bypass	Commit Rejected [38]
Link Validator	Verifies internal markdown links	Navigation Drift	Commit Rejected [48]

Source: Synthesis of [32]

These hooks enforce the **Pointer-Only** architecture. If a developer (or agent) attempts to commit a file that breaks a link in llms.txt, the commit is physically blocked. This preserves the integrity of the agent's context map.[48]

5.2 The "Hard Constraints" CI Pipeline

While Git hooks handle local hygiene, the Continuous Integration (CI) pipeline enforces the "Hard Constraints" that define the system's safety boundaries.

- **Constraint 1: Deterministic Build.** The agent's code must build in a clean, isolated

container. This prevents "works on my machine" drift.[2]

- **Constraint 2: Semantic Verification.** The pipeline runs the semantic linter (e.g., Vale) against the *entire* documentation set. It checks not just for style, but for consistency with the ADRs. If the code implements a pattern that contradicts an active ADR, the build fails.[49]
- **Constraint 3: Infrastructure Drift Check.** Tools like Terraform or specialized drift detection agents scan the deployed infrastructure and compare it to the repository code. If drift is detected, the pipeline halts deployment and triggers a remediation workflow.[2]

5.3 Semantic Linting with Vale

The nukeCE doctrine mandates the use of **Vale**, a prose linter that allows for customizable style rules.[47] Unlike a spell-checker, Vale can be configured to enforce semantic rules.

- **Consistency Checks:** Vale ensures that terms are used consistently (e.g., enforcing "Repo Map" over "Repository Graph").
- **Duplication Detection:** Custom Vale rules can detect when a paragraph is repeated across multiple files, flagging it as a violation of the "One Authoritative Place" principle.[47]
- **Bias Enforcement:** Vale can also enforce the "Soft Preferences" defined in AGENTS.md (e.g., ensuring a specific tone or terminology), effectively treating the documentation quality as a build artifact.[50]

6. Lifecycle Management: The "Nuke" Protocol

The final component of the doctrine is the rigorous lifecycle management of the data itself. Entropy accumulates when obsolete data is allowed to linger. The nukeCE doctrine implements a "Nuke" protocol for data deletion.

6.1 Soft Deletes are for Humans, Hard Deletes for Agents

In traditional database management, "soft delete" (marking a record as is_deleted=true but keeping it) is common practice.[52] However, for AI agents, soft-deleted files are dangerous. They persist in the repository, potentially being indexed by search tools or included in broad context sweeps, leading to "Zombie Data" hallucinations.[53]

Doctrine Principle 6.1: *The agent must operate in a Clean Room. Obsolete data must be Hard Deleted from its view.*

- **Mechanism:** When a file is deprecated, it is not merely marked. It is moved to a distinct archive/ directory.[54]
- **Context Exclusion:** The archive/ directory is explicitly excluded from the AI's context window via .gitignore and the llms.txt exclusion rules. This effectively "hard deletes" the file from the AI's perspective—it ceases to exist in the context window—while preserving it for human audit.[54]

- **The "Nuke" Threshold:** After a defined retention period (e.g., 90 days), the archived data undergoes **Secure Deletion**. It is cryptographically erased to ensure it cannot be recovered or re-indexed. This is crucial for "Trash Intelligence" security, preventing the AI from recovering sensitive data (like old API keys) from the git history.[38]

6.2 The Lifecycle of a Repository File

The nukeCE doctrine defines a strict lifecycle for every file in the repository to prevent sprawl.

1. **Creation (Ingestion):** A new file is created. It must immediately be registered in the llms.txt (if high-value) or excluded (if low-value). The repomap is updated to include it.[24]
2. **Maintenance (Validation):** On every commit, the file is subject to semantic linting and link validation.
3. **Deprecation (The signal):** When the file becomes obsolete, an ADR is updated to mark the feature "Deprecated."
4. **Archival (The Soft Nuke):** The file is moved to archive/ and removed from llms.txt. The agent no longer sees it.
5. **Deletion (The Hard Nuke):** The file is permanently purged from the repository history.[38]

7. Case Study: The Doctrine in Action

To illustrate the nukeCE doctrine, consider a scenario where an organization swaps its primary AI model from GPT-4 to Claude 3.5 and simultaneously refactors a core microservice.

Scenario: The team updates AGENTS.md to reflect the new "Soft Preferences" for Claude 3.5 (e.g., adjusting the prompt style for XML tags).

1. **Interchangeability:** The new model reads AGENTS.md and immediately understands the repository constraints. It does not need "retraining" on the repo's rules; it just reads the constitution.[36]
2. **Context Loading:** The agent reads llms.txt. It sees the "Core Logic" section and requests the repomap.
3. **Action:** The agent proposes a refactor. It consults the ADR log and sees that "Statelessness" is a Hard Constraint.
4. **Enforcement:** The agent writes code. It attempts to create a new utils.py that duplicates logic from helpers.py.
5. **Rejection:** The **Semantic Linter** (Vale) in the pre-commit hook detects the semantic duplication. The commit is blocked. The agent is instructed to link to helpers.py instead.
6. **Success:** The agent corrects the code, uses a pointer, and the commit passes. The repomap is automatically regenerated to reflect the new structure.

8. Conclusion: The Immutable Doctrine

The nukeCE Anti-Drift Doctrine redefines the relationship between the repository and the AI agent. It asserts that **ambiguity is damage**. By enforcing a Single Source of Truth through

pointer-only architectures, distinguishing soft preferences from hard permissions, and rigorously managing the lifecycle of context, we create a repository that is hostile to entropy.

In this system, the AI agent is not an oracle but a highly constrained operator. It works within a "clean room" environment where the Context Map (llms.txt) is the map, the AGENTS.md file is the handbook, and the Git hooks are the guards. This structure ensures that as models become more powerful and interchangeable, the repository remains the immutable anchor of truth, resilient to the drift that plagues less disciplined systems. The "nuke" protocol ensures that the past does not haunt the future, keeping the agent's attention focused solely on the coherent, validated reality of the present.

Works cited

1. Understanding and Managing Configuration Drift - OpsLevel, accessed January 15, 2026, https://www.opslevel.com/resources/understanding-and-managing-configuration-drift
2. Detection and Management of Configuration Drifts & Secrets Exposure - Entro Security, accessed January 15, 2026, https://entro.security/blog/detection-management-configuration-drifts/
3. Architecture Drift: What It Is and How It Leads to Breaches - CrowdStrike, accessed January 15, 2026, https://www.crowdstrike.com/en-us/blog/architecture-drift/
4. Know When To Stop: A Study of Semantic Drift in Text Generation - ACL Anthology, accessed January 15, 2026, https://aclanthology.org/2024.naacl-long.202.pdf
5. Know When To Stop: A Study of Semantic Drift in Text Generation - arXiv, accessed January 15, 2026, https://arxiv.org/html/2404.05411v1
6. Manage Context Window Size With Advanced AI Agents | Octopus blog, accessed January 15, 2026, https://octopus.com/blog/advanced-ai-agents
7. Context Window Management: Strategies for Long-Context AI Agents and Chatbots, accessed January 15, 2026, https://www.getmaxim.ai/articles/context-window-management-strategies-for-long-context-ai-agents-and-chatbots/
8. How Context Engineering Is Shaping the Future of AI Agents | Medium, accessed January 15, 2026, https://masteringllm.medium.com/how-context-engineering-is-shaping-the-future-of-ai-agents-2607640219f3
9. How to Optimize Context Windows So Your AI Agents Handle Complex Information, accessed January 15, 2026, https://datagrid.com/blog/optimize-ai-agent-context-windows-attention
10. Documentation as Code: How Agile Development Implements the One Source of Truth Philosophy - SCAND, accessed January 15, 2026, https://scand.com/company/blog/documentation-as-code-agile-one-source-of-truth/

11. What is Docs as Code? Your Guide to Modern Technical Documentation - Kong Inc., accessed January 15, 2026, https://konghq.com/blog/learning-center/what-is-docs-as-code
12. Managing Digital Twins in the Construction Sector with an Extended Catalog Service - mediaTUM, accessed January 15, 2026, https://mediatum.ub.tum.de/doc/1839038/8cdao8dra7xq69ux5f8v55ib5.Ying_Lu_Thesis.pdf
13. Lec03 Arrays, Pointers Strings | PDF | Pointer (Computer Programming) | Integer (Computer Science) - Scribd, accessed January 15, 2026, https://www.scribd.com/document/819718630/Lec03-Arrays-Pointers-Strings
14. Organize Your Projects Better with Symlinks (and Save Time) - DEV Community, accessed January 15, 2026, https://dev.to/rijultp/organize-your-projects-better-with-symlinks-and-save-time-5gh5
15. CODE CLONE DETECTION WITH SELF-SUPERVISION ON DUAL GRAPHS, accessed January 15, 2026, https://oresta.org/menu-script/index.php/oresta/article/download/845/297
16. Mirror Code: Uncovering Hidden Duplicates Across Systems - IN-COM DATA SYSTEMS, accessed January 15, 2026, https://www.in-com.com/blog/mirror-code-uncovering-hidden-duplicates-across-systems/
17. SemDeDup: Data-efficient learning at web-scale through semantic deduplication, accessed January 15, 2026, https://openreview.net/forum?id=IRSesTQUtb¬eId=usQjFYYAZJ
18. Semantic Deduplication - NeMo-Curator - NVIDIA Documentation, accessed January 15, 2026, https://docs.nvidia.com/nemo/curator/25.09/curate-text/process-data/deduplication/semdedup.html
19. Semantic Content Cannibalization Detection — Identifying and Resolving Overlapping Topics Across Pages - ThatWare, accessed January 15, 2026, https://thatware.co/semantic-content-cannibalization-detection/
20. How to find semantic duplicates in requirements documents - Qualicen, accessed January 15, 2026, https://www.qualicen.de/semantic-smells/
21. Architectural Decision Record (ADR) | Mia-Platform Blog, accessed January 15, 2026, https://mia-platform.eu/blog/architectural-decision-record-adr/
22. Architecture Decision Records: How we make better technical choices at MakerX, accessed January 15, 2026, https://blog.makerx.com.au/architecture-decision-records-how-we-make-better-technical-choices-at-makerx/
23. Architecture decision record (ADR) examples for software planning, IT leadership, and template documentation - GitHub, accessed January 15, 2026, https://github.com/joelparkerhenderson/architecture-decision-record
24. AGENTS.md - Factory Documentation, accessed January 15, 2026, https://docs.factory.ai/cli/configuration/agents-md
25. LLMs.txt: The Emerging Standard Reshaping AI-First Content Strategy |

ScaleMath, accessed January 15, 2026, https://scalemath.com/blog/llms-txt/
26. llms-txt: The /llms.txt file, accessed January 15, 2026, https://llmstxt.org/
27. LLMs txt Vs Robots txt What Is The Difference - Cension AI, accessed January 15, 2026, https://cension.ai/blog/llms-txt-vs-robots-txt-what-is-the-difference/
28. llms.txt isn't robots.txt: It's a treasure map for AI - Search Engine Land, accessed January 15, 2026, https://searchengineland.com/llms-txt-isnt-robots-txt-its-a-treasure-map-for-ai-456586
29. LLMs.txt: The New Standard for AI Crawling - Overdrive Interactive, accessed January 15, 2026, https://www.ovrdrv.com/insights/llms-txt-the-new-standard-for-ai-crawling
30. LLMS.txt Best Practices & Implementation Guide - Rankability, accessed January 15, 2026, https://www.rankability.com/guides/llms-txt-best-practices/
31. What is llms.txt? Breaking down the skepticism - Mintlify, accessed January 15, 2026, https://www.mintlify.com/blog/what-is-llms-txt
32. Repository map - Aider, accessed January 15, 2026, https://aider.chat/docs/repomap.html
33. RepoMapper: Your AI's GPS for Complex Codebases, accessed January 15, 2026, https://skywork.ai/skypage/en/repomapper-ai-gps-codebases/1980849506976722944
34. Building a better repository map with tree sitter - Aider, accessed January 15, 2026, https://aider.chat/2023/10/22/repomap.html
35. AGENTS.md Files | Kilo Code Docs, accessed January 15, 2026, https://kilo.ai/docs/agent-behavior/agents-md
36. AGENTS.md — a simple, open format for guiding coding agents - GitHub, accessed January 15, 2026, https://github.com/agentsmd/agents.md
37. DeepSom: a CNN-based approach to somatic variant calling in WGS samples without a matched normal - Oxford Academic, accessed January 15, 2026, https://academic.oup.com/bioinformatics/article/39/1/btac828/6986966
38. Trash Intelligence: Investigating How AI Exploits Deleted Data and Digital Dustbins in Cybersecurity - IJCRT.org, accessed January 15, 2026, https://ijcrt.org/papers/IJCRT2508545.pdf
39. CODS : A Theoretical Model for Computational Design Based on Design Space - arXiv, accessed January 15, 2026, https://arxiv.org/html/2506.18455v1
40. 10 Advanced Prompting Techniques. A standardized playbook for teams that... | by cLEarn Project | Dec, 2025 | Medium, accessed January 15, 2026, https://medium.com/@cLEarn/10-advanced-prompting-techniques-07cd43c096ff
41. TPC@IJCAI2025 - GitHub Pages, accessed January 15, 2026, https://chinatravel-competition.github.io/IJCAI2025/
42. What is Context Mapping? - Dremio, accessed January 15, 2026, https://www.dremio.com/wiki/context-mapping/
43. ContextGit/USER_GUIDE.md at main - GitHub, accessed January 15, 2026, https://github.com/Mohamedsaleh14/ContextGit/blob/main/USER_GUIDE.md
44. System Prompts and Configuration - The Agentic Systems Series, accessed

January 15, 2026, https://gerred.github.io/building-an-agentic-system/system-prompts-and-model-settings.html
45. Git Hooks, accessed January 15, 2026, https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks
46. Git commit resulting in a missing tree error - Stack Overflow, accessed January 15, 2026, https://stackoverflow.com/questions/9674684/git-commit-resulting-in-a-missing-tree-error
47. Word Repetition Check Vale Rule - Hyperlint Resources, accessed January 15, 2026, https://hyperlint.com/vale/rule/word-repetition-check/
48. LLMs.txt Validator - Validate Your AI Config Files Instantly - AccuWeb Hosting, accessed January 15, 2026, https://www.accuwebhosting.com/resources/llms-txt-validator
49. Prose linting with Vale - Meilisearch, accessed January 15, 2026, https://www.meilisearch.com/blog/prose-linting-with-vale
50. Elastic style guide for Vale | Elastic Docs, accessed January 15, 2026, https://www.elastic.co/docs/contribute-docs/vale-linter
51. Introduction - Vale CLI, accessed January 15, 2026, https://vale.sh/docs
52. What is the best way to implement soft deletion? - Stack Overflow, accessed January 15, 2026, https://stackoverflow.com/questions/68323/what-is-the-best-way-to-implement-soft-deletion
53. Are soft deletes a good idea? [duplicate] - Stack Overflow, accessed January 15, 2026, https://stackoverflow.com/questions/2549839/are-soft-deletes-a-good-idea
54. Repository storage - GitLab Docs, accessed January 15, 2026, https://docs.gitlab.com/administration/repository_storage_paths/
55. Use soft-deleted objects - Storage - Google Cloud Documentation, accessed January 15, 2026, https://docs.cloud.google.com/storage/docs/use-soft-deleted-objects